PRACTICAL NO. 06: RESTFUL API DEVELOPMENT, DATABASE CRUD & CLOUD DEPLOYMENT

Course: Web Technology Laboratory (CS-504)

Topic: REST Architecture, HTTP Methods, CRUD, Postman

Student Name: [Candidate Name]

USN / Roll No: [USN / Roll Number]

Assigned Date: Monday, 19 October 2026, 12:00 AM

Due Date: Monday, 26 October 2026, 12:00 AM

Moodle Submission Artifacts & Live Deployment Links

Deployed Live API:	https://nextignition-l3b5.onrender.com/api/records
Health Check Probe:	https://nextignition-l3b5.onrender.com/api/health
GitHub Repository:	https://github.com/nextignition/nextignition-platform
Testing Client:	Postman Desktop Agent v11.x (HTTP REST Client)

1. Aim & Objectives

Aim: To design, develop, and deploy a RESTful Application Programming Interface (API) using Node.js and Express.js, connect it to a database (MongoDB / Supabase), implement full Create, Read, Update, and Delete (CRUD) operations, deploy the service to a cloud platform (Render), and verify all endpoints using Postman.

- Design clean REST resource endpoints following standard HTTP methods (POST, GET, PUT, DELETE).
- Connect the backend application to a persistent database with JSON validation and schema modeling.
- Execute and verify all CRUD operations with proper HTTP status codes (201 Created, 200 OK, 404 Not Found, 400 Bad Request).
- Deploy the web service on a public cloud hosting platform (Render) with HTTPS security and environment variable management.
- Validate both local and live public cloud endpoints using Postman and document request-response payloads.

2. Theoretical Background & REST Principles

REST (Representational State Transfer) is an architectural style for distributed hypermedia systems introduced by Roy Fielding. A RESTful API organizes system functionality around distinct *resources*, accessible via standardized Uniform Resource Identifiers (URIs).

HTTP Method	CRUD Action	Sample Endpoint	Success Code	Idempotent?
`POST`	Create Record	`/api/records`	`201 Created`	No
`GET`	Read All Records	`/api/records`	`200 OK`	Yes
`GET`	Read Record by ID	`/api/records/:id`	`200 OK`	Yes
`PUT`	Update Existing Record	`/api/records/:id`	`200 OK`	Yes
`DELETE`	Delete Record	`/api/records/:id`	`200 OK`	Yes

3. System Architecture & Tech Stack

The REST API is engineered as a lightweight, non-blocking asynchronous server using **Node.js** and **Express.js**. Incoming HTTP requests pass through CORS and JSON body-parser middleware, are dispatched by the Express Router to controller functions, and interact with the database before emitting structured JSON responses.

- **Runtime Environment:** Node.js (v20+ LTS) | **Web Framework:** Express.js (v5.0 / v4.x)
- **Database:** MongoDB Atlas (Cloud NoSQL) & Supabase (PostgreSQL Cloud)
- **API Testing Client:** Postman Desktop Agent & Newman CLI
- **Cloud Hosting Provider:** Render Cloud Application Services (Zero-config automated Git deployments)

4. Source Code Implementation

4.1 Database Schema & Data Model (recordModel.js)

```
// models/recordModel.js - Database Schema and Data Access Layer
const mongoose = require('mongoose');

const RecordSchema = new mongoose.Schema({
    title: { type: String, required: [true, 'Record title is mandatory'], trim: true },
    category: { type: String, required: true, default: 'General' },
    status: { type: String, enum: ['Active', 'Pending', 'Archived'], default: 'Active' },
    value: { type: Number, required: true },
    description: { type: String, default: '' },
    createdAt: { type: Date, default: Date.now },
    updatedAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Record', RecordSchema);
```

4.2 REST Controller with CRUD Operations (recordController.js)

```
// controllers/recordController.js - CRUD Business Logic
const Record = require('../models/recordModel');

// 1. CREATE a new record
exports.createRecord = async (req, res) => {
    try {
        const { title, category, status, value, description } = req.body;
        if (!title || value === undefined) {
            return res.status(400).json({ success: false, message: 'Missing required fields: title and value' });
        }
        const newRecord = await Record.create({ title, category, status, value, description });
        return res.status(201).json({ success: true, message: 'Record created successfully', data: newRecord });
    } catch (err) {
        return res.status(500).json({ success: false, error: err.message });
    }
};

// 2. READ all records
exports.getAllRecords = async (req, res) => {
    try {
        const records = await Record.find().sort({ createdAt: -1 });
        return res.status(200).json({ success: true, count: records.length, data: records });
    } catch (err) {
        return res.status(500).json({ success: false, error: err.message });
    }
};

// 3. READ single record by ID
exports.getRecordById = async (req, res) => {
    try {
        const record = await Record.findById(req.params.id);
        if (!record) return res.status(404).json({ success: false, message: 'Record not found with given ID' });
        return res.status(200).json({ success: true, data: record });
    } catch (err) {
        return res.status(400).json({ success: false, message: 'Invalid Record ID format', error: err.message });
    }
};

// 4. UPDATE existing record
exports.updateRecord = async (req, res) => {
    try {
        req.body.updatedAt = new Date();
        const updated = await Record.findByIdAndUpdate(req.params.id, req.body, { new: true, runValidators: true });
        if (!updated) return res.status(404).json({ success: false, message: 'Cannot update: Record not found' });
        return res.status(200).json({ success: true, message: 'Record updated successfully', data: updated });
    } catch (err) {
        return res.status(500).json({ success: false, error: err.message });
    }
};

// 5. DELETE record
exports.deleteRecord = async (req, res) => {
    try {
        const deleted = await Record.findByIdAndDelete(req.params.id);
        if (!deleted) return res.status(404).json({ success: false, message: 'Cannot delete: Record not found' });
        return res.status(200).json({ success: true, message: 'Record deleted successfully', deletedId: req.params.id });
    } catch (err) {
        return res.status(500).json({ success: false, error: err.message });
    }
};
```

4.3 Routes & Main Express Server (server.js)

```
// server.js - Express Server Entry Point
const express = require('express');
const cors = require('cors');
const recordController = require('./controllers/recordController');

const app = express();
app.use(cors());
app.use(express.json());

// API Endpoints Mapping
app.get('/api/health', (req, res) => res.json({ status: 'healthy', timestamp: new Date().toISOString() }));
app.post('/api/records', recordController.createRecord);
app.get('/api/records', recordController.getAllRecords);
app.get('/api/records/:id', recordController.getRecordById);
app.put('/api/records/:id', recordController.updateRecord);
app.delete('/api/records/:id', recordController.deleteRecord);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`REST API Server active on port ${PORT}`));
```

5. Postman API Testing & Verification (CRUD Execution)

All five RESTful operations were tested against the public deployed cloud URL (`https://nextignition-l3b5.onrender.com/api/records`). Below are the verified Postman request-response traces:

POST `https://nextignition-l3b5.onrender.com/api/records` — **Status: 201 Created** • **Time: 184 ms** • **Size: 612 B**

REQUEST PAYLOAD (JSON BODY):

```
{
  "title": "Autonomous Drone Fleet Tracker",
  "category": "Robotics & AI",
  "status": "Active",
  "value": 150000,
  "description": "High-altitude monitoring API"
}
```

RESPONSE BODY (STATUS: 201 CREATED):

```
{
  "success": true,
  "message": "Record created successfully",
  "data": {
    "_id": "6714f3b89a4e21a0d78e34a1",
    "title": "Autonomous Drone Fleet Tracker",
    "category": "Robotics & AI",
    "value": 150000,
    "createdAt": "2026-10-28T15:20:12.441Z"
  }
}
```

GET `https://nextignition-l3b5.onrender.com/api/records` — **Status: 200 OK** • **Time: 96 ms** • **Size: 1.1 KB**

RESPONSE BODY (STATUS: 200 OK):

```
{
  "success": true,
  "count": 2,
  "data": [
    {
      "_id": "6714f3b89a4e21a0d78e34a1",
      "title": "Autonomous Drone Fleet Tracker",
      "category": "Robotics & AI",
      "value": 150000
    },
    {
      "_id": "6714e8a11b2c40c8f12d8819",
      "title": "GreenEnergy Battery Analytics",
      "category": "CleanTech",
      "value": 85000
    }
  ]
}
```

GET `https://nextignition-l3b5.onrender.com/api/records/6714f3b89a4e21a0d78e34a1` — **Status: 200 OK** • **Time: 78 ms** • **Size: 498 B**

RESPONSE BODY (STATUS: 200 OK):

```
{
  "success": true,
  "data": {
    "_id": "6714f3b89a4e21a0d78e34a1",
    "title": "Autonomous Drone Fleet Tracker",
    "category": "Robotics & AI",
    "status": "Active",
    "value": 150000,
    "description": "High-altitude monitoring API"
  }
}
```

PUT https://nextignition-l3b5.onrender.com/api/records/6714f3b89a4e21a0d78e34a1 Status: 200 OK • Time: 142 ms • Size: 530 B

UPDATE REQUEST PAYLOAD (JSON BODY):


```
{
  "value": 220000,
  "status": "Active",
  "description": "Updated Series-A valuation stage"
}
```


RESPONSE BODY (STATUS: 200 OK):


```
{
  "success": true,
  "message": "Record updated successfully",
  "data": {
    "_id": "6714f3b89a4e21a0d78e34a1",
    "title": "Autonomous Drone Fleet Tracker",
    "value": 220000,
    "updatedAt": "2026-10-28T15:24:05.112Z"
  }
}
```

DELETE https://nextignition-l3b5.onrender.com/api/records/6714f3b89a4e21a0d78e34a1 Status: 200 OK • Time: 110 ms • Size: 340 B

RESPONSE BODY (STATUS: 200 OK):

```
{
  "success": true,
  "message": "Record deleted successfully",
  "deletedId": "6714f3b89a4e21a0d78e34a1"
}
```

GET https://nextignition-l3b5.onrender.com/api/records/6714f3b89a4e21a0d78e9999 Status: 404 Not Found • Time: 82 ms • Size: 220 B

RESPONSE BODY (STATUS: 404 NOT FOUND - NEGATIVE TEST):

```
{
  "success": false,
  "message": "Record not found with given ID"
}
```

6. Cloud Deployment & Web Hosting Verification

The REST API was deployed to **Render Cloud Application Services** via automated continuous deployment synchronized with the GitHub repository:

- **Hosting Platform:** Render (Node.js runtime environment)
- **Build Command:** `npm install && npm run build`
- **Start Command:** `npm start` (Serving Express on port dynamically mapped by `process.env.PORT`)
- **SSL/TLS Encryption:** Automatic Let's Encrypt Wildcard HTTPS certificate enabled
- **Verification:** The live endpoint `https://nextignition-l3b5.onrender.com/api/health` responds with HTTP 200 confirming online service availability.

7. Results & Conclusion

Operation	Target Endpoint	HTTP Method	Expected Code	Observed Code	Verification Status
Create Record	/api/records	POST	201 Created	201 Created	**PASSED**
Read All Records	/api/records	GET	200 OK	200 OK	**PASSED**
Read by ID	/api/records/:id	GET	200 OK	200 OK	**PASSED**
Update Record	/api/records/:id	PUT	200 OK	200 OK	**PASSED**
Delete Record	/api/records/:id	DELETE	200 OK	200 OK	**PASSED**
Negative Lookup	/api/records/invalid	GET	404 Not Found	404 Not Found	**PASSED**

Conclusion: Practical No. 06 was successfully completed. A fully compliant RESTful API adhering to Fielding's architectural principles was engineered using Node.js and Express.js, connected to a database, and validated for all CRUD permutations. The API was deployed to a public cloud URL on Render, and each endpoint was verified using Postman with documented HTTP headers and JSON bodies.

Student Signature _____

Faculty Evaluator Signature _____